

Remember: this is an oral presentation homework! You are required to work in groups of three, and may collaborate only with other students in your group. We encourage you to write up solutions, but your grade will be based entirely on your presentation. At your presentation, the grader will randomly assign each student to present a problem.

You may use the internet or LLMs to look up formulas, definitions, concepts, etc., but may not simply look up the answers online or ask an LLM.

1 Dumbbell Matching (33 points)

You belong to a gym which has two sets of dumbbells A and B , each of which has n dumbbells. You know that there is a correspondence between the sets: for every dumbbell in set A there is exactly one dumbbell in set B that has the same weight, and similarly for every dumbbell in set B there is exactly one dumbbell in set A that has the same weight. You want to perform exercises that require two dumbbells of the same weight. So you want to pair up the dumbbells by weight, i.e., for every dumbbell you want to know which dumbbell from the other set has the exact same weight.

Unfortunately the dumbbells are unsorted and unlabeled, and you can't tell their weights by looking at them. The only way to compare two dumbbells is to pick them both up simultaneously (one in each hand) and perform a curl. By comparing the strain on your arms, you can tell whether the two dumbbells are the same weight, and if not, which one is heavier. Even more unfortunately, the owner of the gym has a rule that two dumbbells from the same set cannot be used at the same time. So you can compare a dumbbell from set A to a dumbbell from set B , but cannot compare two dumbbells from the same set.

Prove that any deterministic algorithm which solves the problem must make at least $\Omega(n \log n)$ comparisons.

In this problem, we must compare 2 sets of n elements to find each dumbbell's pair. This is a comparison-based

2 Group Sorting (33 points)

In the last homework you designed an algorithm for k -group-sorting in time $O(n \log k)$. Prove a matching lower bound in the comparison model. That is, prove that any algorithm for k -group-sorting in the comparison model must make at least $\Omega(n \log k)$ comparisons in the worst case.

Let us consider the number of permutations of array A to determine the number of comparisons needed for k -group sorting. We assume A has n elements, and we are trying to sort it into k groups. On the whole, since there are n elements, there are $n!$ permutations of A . Now, let us calculate the number of permutations of A for which $n!$ is k -group sorted. Since the array is broken up in k groups, each group would have $\frac{n}{k}$ elements in it. Therefore, within each group, there are $\frac{n}{k}!$ permutations. There are k groups, and we must assume they are in order, for A to be

k -group-sorted. Therefore, the number of permutations of A such that A is k -group sorted would be $k * \frac{n!}{k!}$.

3 Sorting Different-Length Items (34 points)

We saw in class some fast sorting algorithms where we assumed that the elements were integers with the same number of digits. In this problem we change those assumptions slightly.

Suppose now that we are given an array of strings (over some finite alphabet, say the letters a-z). Each string can have a different number of characters, but the total number of characters in all the strings (i.e., the sum over the strings of the length of the string) is equal to n . Show to sort the strings lexicographically in $O(n)$ time. Here lexicographic order is the standard alphabetic order, so for example $a < ab < b$.

Algorithm Description:

Since the total number of characters in all strings is n , we know that any array A must have at most n elements (in which case every element has 1 character). For this, I consider a tree structure, wherein the A is reconstructed in tree format, with a root node, and each element is represented as characters cascading down a tree. There is a *root* node from which all elements originate, and each ends with a terminator character. For each string, I find its place in the tree by cascading down the tree. Each character is a node. If the node exists in the tree, then there is no need to add it again, and the next character is added as a child of that node. If the node does not exist, its place is found by iterating through the children to find at which point the character fits. Since the alphabet has 26 letters and the algorithm accounts for 1 terminator, there would be at most 27 comparisons, so this sorting would take $O(27)$ comparisons, or $O(1)$. At the end of the string, a terminator character would be added to indicate the end of the string, which would be sorted sequentially first. Since there is nothing in the problem assuming that the set of strings is unique, this null character can keep track of the number of instances of a given string. For example, if 'cat' appears twice in A , the child of Node 't' would first be 1 after the first instance is added to the tree, and would increment to 2 after the second. Since there is a constant number of letters in the alphabet, each operation would take $O(1)$ time. This constructs a tree by following this process for each character in each element in A . Since there are n characters spread throughout A , this would be an $O(1)$ process repeating n times, making the time to build the tree $O(n)$.

From here, we must reconstruct the tree to create a sorted list of characters. We will use recursive backtracking to do so. We traverse the tree in a DFS-type manner each time to construct each character, completing and adding the string to the sorted list. From here, we would backtrack to find the next string in order. Since this algorithm visits every node (that is, every unique character) once, it would visit at most n nodes. Since each node is examined once with a constant number of operations per node, this means that the list reconstruction would take $O(n)$ time.

Therefore, since this algorithm uses two sequential $O(n)$ processes, it would take at most $O(2n)$ time, or, simplified, $O(n)$.

I do want to address the main drawback to this algorithm, which is that each sequential placement of a character as a node could take up to 26 comparisons. The tree creation algorithm does maintain a growth rate of $O(n)$, but it can be less efficient than just a classic $O(n^2)$ or even $O(n \log n)$ sorting algorithm, if $n \leq 27$. However, for very large n , this would be very efficient.

Correctness Proof:

I will prove correctness in two parts. First, I prove that the tree creation does create a tree with each subsequent character in sorted order, and then, I prove that the reconstruction of the list from the tree produces a sorted list of strings.

Runtime Proof:

Once again, I prove runtime in two parts. First, I prove that tree creation takes $O(n)$ time, and then that tree traversal for reconstruction takes $O(n)$ time. Therefore, as these steps happen sequentially, the algorithm as a whole takes $O(n)$ time.

The key observation here is that each node's addition takes at most 27 comparisons, and therefore, has a growth rate of $O(1)$. The correctness proof and algorithm description shows that no node's child can have a repeated character. That is, if strings 'car' and 'cat' are in array A , the character 'a' would not appear twice as a child of node 'c'. As the characters are bounded within $a-z$, as well as a possible terminator digit, this leaves 27 possibilities, and a maximum of 27 children per node. In any case, either when adding a node where the character is already a child in the tree or adding a new child, the algorithm must iterate through the sorted set of children until it finds a position where the character is greater than those to the left and less than those to the right (barring where the character is either the least or the greatest sequentially). Let us consider a worst case scenario to prove that this additive step takes at most 27 steps, regardless of the value of n . Consider Array A , which contains elements "c", "ca", "cb", "cc", ... "cz", as well as a second "cz". As per the correctness proof, once the first 27 elements have been added to the tree, the c node has 26 children. To add the 27th, the second 'z', the algorithm must iterate through each child, checking if 'z' is less than each subsequent character. Since 'z' has the greatest possible value, it would need to iterate through 26 elements, before finding its equal at the 27th, which means there would be 27 total operations in the worst case. If we imagine a version of A with 5 "cz" strings, each subsequent addition of 'z' as a child of 'c' would take 27 comparisons. As this is a constant, bounded value for the worst case, the node placement algorithm can be expressed as $O(1)$ to represent the constant number of operations that do not depend on the size of the input n . This process of adding to the tree repeats for each character in each element of the list. Since there are a total of n characters in the list, this $O(1)$ process repeats n times. Therefore, tree creation takes $O(n)$ time.

Now, I will move on to array reconstruction. Because this algorithm, in its worst case, visits every individual character in each element, along with each element's terminator, this section of the algorithm has a time complexity of $O(n)$. Let us consider a worst case in which no two elements begin with the same set of letters, so each child of the root corresponds to at most one element. This represents the worst case, since in cases where there is some overlap in the letters, the iterative step would visit a total number of nodes less than n , whereas in this worst case, the iterative step visits n nodes. I will walk through the algorithm to prove an $O(n)$ worst-case complexity.